

3_Serwisy

March 28, 2022

1 Obsługa ROS Service

1.1 Używanie komend z terminala

1.1.1 Wprowadzenie

ROS Service jest kolejnym sposobem komunikacji. Zaletą serwisów jest możliwość wysłania zapytania do serwera. Podobnie jak dla ROS topic należy znać format wiadomości. Serwisy w danej paczce ROS przechowywane są w katalogu `srv`, a rozszerzenie serwisu to `.srv`.

Struktura wiadomości serwisowej Wyróżnia się podział na format wiadomości:

- wysłany przez klienta (górną część wiadomości nad znakiem ---)
- odpowiedź serwera (dolną część wiadomości pod znakiem ---)

Z lewej strony należy podać typ wiadomości ROS. Mogą być one bardziej złożone i składać się z już utworzonych wiadomości (ROS msg). Z prawej strony podawana jest nazwa pola.

```
[3]: # Dla obsługi wiadomości serwisowych używane jest polecenie rossrv. Polecenie ↵  
    ↪można wywołać  
    # z następującymi parametrami:  
    !rossrv --help
```

`rossrv` is a command-line tool for displaying information about ROS Service types.

Commands:

<code>rossrv show</code>	Show service description
<code>rossrv info</code>	Alias for <code>rossrv show</code>
<code>rossrv list</code>	List all services
<code>rossrv md5</code>	Display service md5sum
<code>rossrv package</code>	List services in a package
<code>rossrv packages</code>	List packages that contain services

Type `rossrv <command> -h` for more detailed usage

Typ serwisu to **nazwa paczki + nazwa serwisu.srv**. Wyświetlenie przykładowego serwisu znajdującego się w paczce `tsr_materials`:

```
[1]: !rossrv show pkg_tsr/Task
```

```
float64 x
float64 y
---
string result
```

```
[4]: !rossrv info pkg_tsr/Task
```

```
float64 x
float64 y
---
string result
```

Wyświetlanie dostępnej pomocy dla polecenia `rosservice`.

```
[7]: !rosservice --help
```

Commands:

```
rosservice args print service arguments
rosservice call call the service with the provided args
rosservice find find services by service type
rosservice info print information about service
rosservice list list active services
rosservice type print service type
rosservice uri print service ROSRPC uri
```

Type `rosservice <command> -h` for more detailed usage, e.g. `'rosservice call -h'`

Wyświetlanie listy aktualnie dostępnych serwisów

```
[6]: # Dostępne serwisy wraz z uruchomioną symulacją turtlesim
!rosservice list
```

```
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
```

```
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```

1.1.2 Dostępne serwisy dla turtlesim node

W podstawowym zakresie dostępne są następujące serwisy bez względu na robota:

- /clear - wyczyszczenie narysowanych ścieżek
- /kill - usunięcie robota
- /reset - reset środowiska do stanu początkowego
- /spawn - utworzenie nowego robota

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie **turtle1** dostępne są następujące serwisy:

- /turtle1/set_pen - ustawienie koloru pędzla do rysowania
- /turtle1/teleport_absolute - natychmiastowe przeniesienie robota do wskazanej lokalizacji
- /turtle1/teleport_relative - przeniesienie robota, współrzędne podawane w układzie robota

```
[8]: # Dostępne parametry dla serwisu tworzącego nowego żółwia
!rosservice args /spawn
```

x y theta name

Wyświetlenie informacji o serwisie "clear". Informuje o nazwie noda z którego pochodzi serwis, typie wiadomości i przyjmowanych argumentach.

```
[ ]: !rosservice info clear
```

1.1.3 Wywołanie serwisu

Do wywołania serwisu używamy rosservice call a następnie w kolejności podajemy argumenty

rosservice call argument1 argument2 ...

Uwaga! Współrzędne x,y powinny być z przedziału <0,11>

Tworzenie nowego robota.

```
[ ]: !rosservice call spawn 15 15 0 t5
```

Z poziomu terminala można wpisać "rosservice call spawn" i kliknąć kilka razy tabulator, a wiadomość wraz z argumentami powinna uzupełnić się domyślnymi parametrami.

rosservice call spawn "x: 0.0

y: 0.0

theta: 0.0

name: ""

1.1.4 Sprawdzenie typu wiadomości serwisowej

```
[35]: !rosservice type spawn
```

turtlesim/Spawn

1.2 Używanie serwisów w Pythonie - klient

```
[37]: import rospy
rospy.init_node("serwis_node_test")
```

Serwis `/clear` do czyszczenia mapy, bez argumentów. Po stronie klienta do obsługi serwisu używamy `ServiceProxy` z biblioteki `rospy`. Jako pierwszy argument podawana jest nazwa serwisu z którego ma być odebrana odpowiedź, a jako drugi argument podawany jest typ serwisu.

```
[38]: #import typu wiadomości serwisowej
from std_srvs.srv import Empty
```

```
[40]: # Utworzenie klienta "ServiceProxy"
clear_map = rospy.ServiceProxy('clear', Empty)
# Wysłanie zapytania przez klienta. Dla pustego zapytania argument nie jest
# przekazywany do funkcji
clear_map()

# To samo wysłanie zapytania przez klienta tylko z utworzeniem pustego zapytania
# Następuje dodatkowo dodanie do importu EmptyRequest <- zapytanie wysyłane
↳ przez klienta
from std_srvs.srv import EmptyRequest
request = EmptyRequest()
clear_map(request)
```

```
[40]:
```

```
[42]: # Czyszczenie przestrzeni z robotów */reset*
reset_sim_state = rospy.ServiceProxy('reset', Empty)
reset_sim_state()
```

```
[42]:
```

Używanie serwisu z argumentami na przykładzie `/spawn`. Kolejne argumenty polecenia podajemy po przecinku.

```
[ ]: from turtlesim.srv import Spawn
```

```
[ ]: create_new_robot = rospy.ServiceProxy('spawn', Spawn)
# Dla serwisu Spawn konieczne jest podanie kolejnych argumentów. Można je podać
→ jako kolejne
# argumenty w trakcie wysłania zapytania
create_new_robot(3, 3, 0, "t8")

# lub można utworzyć wiadomość z zapytaniem
from turtlesim.srv import SpawnRequest
request = SpawnRequest()
request.x = 7
request.y = 10
request.theta = 3.14
request.name = "nowy2"
create_new_robot(request)
```

```
[ ]: # Zamiast oddzielny importów Spawn i SpawnRequest można zapisać
from turtlesim.srv import Spawn, SpawnRequest
# lub można zaimportować wszystkie wiadomości serwisowe z paczki przy użyciu *
from turtlesim.srv import *
```

1.3 Używanie serwisów w Pythonie - serwer

```
[ ]: # czyszczenie symulacji
!rosservice call reset
```

```
[45]: from pkg_tsr.srv import Task
from turtlesim.srv import TeleportAbsolute
from std_srvs.srv import Empty
```

```
[46]: # Zapis I
def draw_square_function(req):
    # req - przekazany argument, dane wysłane przez klienta, zapytanie typu
    → TaskRequest
    width = 5
    print("Init pose x=%s, y=%s" % (req.x, req.y))
    # utworzenie niezbędnych klientów do teleportacji żółwia i narysowania
    → kwadratów
    set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
    clear_map = rospy.ServiceProxy('clear', Empty)
    set_pose(req.x, req.y, 0)
    clear_map()
    set_pose(req.x, req.y + width, 0)
    set_pose(req.x + width, req.y + width, 0)
```

```

set_pose(req.x + width, req.y,0)
set_pose(req.x, req.y,0)
# dla wiadomości serwisowej typu Task odpowiedź (response) jest typu string,
→dlatego w
# return zapisano od razu wartość tekstową
return "finished"

```

```

[ ]: # Zapis II (analogiczny do powyższego tylko tworzona jest wiadomość z odpowiedzią
def draw_square_function(req):
    # req - przekazany argument, dane wysłane przez klienta, zapytanie typu
    →TaskRequest
    width = 5
    print("Init pose x=%s, y=%s" % (req.x, req.y))
    # utworzenie niezbędnych klientów do teleportacji żółwia i narysowania
    →kwadratów
    set_pose = rospy.ServiceProxy('/turtle1/teleport_absolute', TeleportAbsolute)
    clear_map = rospy.ServiceProxy('clear', Empty)
    set_pose(req.x, req.y, 0)
    clear_map()
    set_pose(req.x, req.y + width, 0)
    set_pose(req.x + width, req.y + width, 0)
    set_pose(req.x + width, req.y, 0)
    set_pose(req.x, req.y, 0)
    # dla wiadomości serwisowej typu Task odpowiedź (response) jest typu string,
    →dlatego w
    # return zapisano od razu wartość tekstową
    response = TaskResponse()
    response.result = "finished"
    return response

```

Po stronie serwera używamy obiektu *Service* z biblioteki *rospy*. W kolejności podajemy 3 następujące argumenty:

- nazwa serwisu
- typ serwisu
- nazwa funkcji, która ma zostać wywołana po pojawieniu się żądania od klienta na tym serwisie

```

[47]: # obiekt serwera. Po wysłaniu zapytania (typTask) od klienta na serwer o nazwie
# robot_teleport wywołana zostanie funkcja draw_square_function
s = rospy.Service('robot_teleport', Task, draw_square_function)

```

UWAGA Wyłączenie serwisu - gdy wprowadzone zostaną jakieś zmiany w funkcji serwisowej, aby nie restartować Kernela Jupyter Notebook, można poniższą metodą `shutdown()` zatrzymać działający serwis.

```

[ ]: s.shutdown()

```

Wynik działania powyższego serwisu po wywołaniu klienta

```
[48]: draw_square = rospy.ServiceProxy('robot_teleport', Task)
      resp = draw_square(3, 2)
```

Init pose x=3.0, y=2.0

Odczyt otrzymanej wartości z serwera

```
[49]: resp.result
```

```
[49]: 'finished'
```

1.4 Dodatek

Tworzenie własnych wiadomości serwisowych. W paczce znajduje się katalog `srv` w którym można utworzyć dodatkowe wiadomości serwisowe. Konfiguracja w przedstawionym dodatku ogranicza się do używania typów wiadomości z poniższego linku:

http://docs.ros.org/en/melodic/api/std_msgs/html/index-msg.html

```
[ ]: !rossrv show pkg_tsr/Task
```

```
[ ]: !rosmmsg show std_msgs/ColorRGBA
```

Przykładowo rozbudowany serwis o wiadomość typu `ColorRGBA` może wyglądać następująco.

```
[ ]: float64 x
      float64 y
      ColorRGBA my_color
      ---
      string result
```

Dodanie nowej wiadomości wymaga zmian w plikach konfiguracyjnych paczki. Ze względu na przyjęte uproszczenie w konfiguracji w paczce `tsr_materials` w pliku **CMakeLists.txt** w miejscu (od 57 linii):

```
[ ]: ## Generate services in the 'srv' folder
      add_service_files(
        FILES
        Task.srv
        Nowy.srv
        #   Service2.srv
      )
```

Należy dopisać nazwę swojego serwisu jak powyżej. Zamiast **Nowy.srv** należy podać nazwę serwisu pod jakim został on zapisany w katalogu `srv`.

```
[ ]: !rossrv show pkg_tsr/... # nazwa utworzonego serwisu, jesli wszystko zostalo  
    ↪prawidlowo utworzone  
    # powinna pojawic sie jego struktura
```

```
[ ]:
```